

Kubernetes Cluster Setup Lab

Building a Multi-Node Cluster on AWS with Terraform, Ansible, kubeadm & Calico

Lab Objective

This lab walks students through provisioning a multi-node Kubernetes cluster on AWS EC2 from scratch — a control node, a master (control-plane) node, and a worker node — using Terraform for infrastructure, Ansible for configuration management, and kubeadm to bootstrap the cluster. Students will also practice real troubleshooting patterns that come up constantly in production and interviews: missing binaries after infrastructure is recreated, broken SSH key paths, container runtime failures, and Kubernetes preflight errors.

Architecture Overview

- Control Node — the machine you run Ansible and kubectl commands from (does not run workloads).
- Master Node (Control Plane) — runs the Kubernetes API server, scheduler, and controller manager.
- Worker Node — joins the cluster and runs application pods.
- Networking — a VPC with a 10.0.0.0/16 (or environment-specific) CIDR block, subnets for each node, and Calico as the pod network (CNI) plugin using 192.168.0.0/16 for pod IPs.

All three nodes are provisioned as AWS EC2 instances via Terraform and connected over SSH using a shared private key.

Prerequisites

- An AWS account with a configured budget alert (recommended: set a billing alarm before starting to avoid unexpected charges).
- Terraform installed locally, with a working main.tf / variables.tf / outputs.tf / provider.tf set for EC2, VPC, and subnet resources.
- A GitHub repository to track infrastructure code (.gitignore should exclude .tfstate files and private keys).
- An SSH key pair (e.g. ansible-lab-key.pem) generated and stored in a known local path.
- Basic familiarity with Linux command line, YAML, and networking concepts (subnets, CIDR, routing).

Part 1 — Provision Infrastructure

1. Set up your AWS account, create a budget alarm, and confirm you're operating within free-tier limits.
2. Initialize your Terraform project and create the VPC, subnets, security groups, and three EC2 instances (control, master, worker).
3. Push your Terraform code to GitHub, excluding state files and secrets.
4. Confirm the VPC and subnets are visible in the AWS console under the CIDR block you defined (e.g. 10.0.0.0/16).

Note: Instance IPs assigned by AWS may not exactly match your planned CIDR ranges if you have multiple subnets or if AWS auto-assigns addresses — always confirm actual IPs via 'hostname -I' rather than assuming from your Terraform variables.

Part 2 — Connect to the Control Node

From your laptop, connect to the control node using the private key:

```
ssh -i /full/path/to/ansible-lab-key.pem ubuntu@<control-node-public-ip>
```

Note: On Windows Git Bash, '~' expands to your Windows user home directory, not wherever you saved the key. If your key lives in a different folder (e.g. C:\Users\aws_keys\), always use the FULL path rather than '~/ansible-lab-key.pem', or the SSH client will fail with 'Identity file not accessible'.

Common pitfall: wrong node

Every node's shell prompt shows its hostname, e.g. ubuntu@ip-10-10-1-142. Before running any Ansible or kubeadm command, verify which node you're on:

```
hostname -I
pwd
```

Ansible commands must be run from the control node — not from the master or worker.

Part 3 — Install Ansible & Build the Inventory

On the control node, install Ansible and create your working directory:

```
sudo apt update
sudo apt install ansible -y
ansible --version
mkdir ~/k8s-lab && cd ~/k8s-lab
```

Create an inventory file listing your master and worker nodes:

```
vi inventory.ini

[k8s_master]
<master-public-ip>

[k8s_worker]
<worker-public-ip>

[k8s_all:children]
k8s_master
k8s_worker

[k8s_all:vars]
ansible_user=ubuntu
ansible_ssh_private_key_file=/home/ubuntu/ansible-lab-key.pem
```

Copy the private key onto the control node so Ansible can use it to reach the other nodes, then test connectivity:

```
ansible all -i inventory.ini -m ping
```

A successful result looks like:

```
<master-ip> | SUCCESS => { "ping": "pong" }
<worker-ip> | SUCCESS => { "ping": "pong" }
```

Part 4 — Install Kubernetes Components via Ansible

Rather than installing kubeadm, kubelet, and kubectl manually on each node, use an Ansible playbook so the setup is repeatable — this matters because recreating EC2 instances (e.g. via 'terraform destroy/apply') wipes all manual installs.

```
vi install-k8s.yml

---
- hosts: k8s_all
  become: yes
  tasks:
    - name: Update packages
      apt:
        update_cache: yes
    - name: Install dependencies
      apt:
        name: [apt-transport-https, ca-certificates, curl, gpg]
        state: present
    - name: Create keyring directory
      file:
```

```

    path: /etc/apt/keyrings
    state: directory
  - name: Add Kubernetes GPG key
    shell: |
      curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.30/deb/Release.key \
      | gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
    args:
      creates: /etc/apt/keyrings/kubernetes-apt-keyring.gpg
  - name: Add Kubernetes repository
    copy:
      dest: /etc/apt/sources.list.d/kubernetes.list
      content: |
        deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg]
        https://pkgs.k8s.io/core:/stable:/v1.30/deb/ /
  - name: Update apt cache
    apt:
      update_cache: yes
  - name: Install Kubernetes packages
    apt:
      name: [kubelet, kubeadm, kubectl]
      state: present
  - name: Hold Kubernetes packages
    dpkg_selections:
      name: "{{ item }}"
      selection: hold
    loop: [kubelet, kubeadm, kubectl]

```

Run the playbook and verify:

```

ansible-playbook -i inventory.ini install-k8s.yml
ansible all -i inventory.ini -m shell -a "kubeadm version"

```

Part 5 — Fix the Container Runtime (containerd)

kubeadm requires a running container runtime. On a freshly recreated instance, containerd is often not installed at all — check first before assuming it's just stopped:

```
sudo systemctl status containerd
```

Note: 'Unit containerd.service could not be found' means containerd isn't installed — not that it's stopped. Install it explicitly rather than trying to just restart the service.

Install and configure containerd on both master and worker:

```

sudo apt update && sudo apt install containerd -y
sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml
sudo systemctl daemon-reload
sudo systemctl enable containerd
sudo systemctl restart containerd
sudo systemctl status containerd    # expect: active (running)

```

Enable IP forwarding, required for pod-to-pod networking:

```

sudo sysctl -w net.ipv4.ip_forward=1
cat /proc/sys/net/ipv4/ip_forward    # expect: 1

```

Note: Setting sysctl this way is temporary (resets on reboot). For a persistent lab setup, add 'net.ipv4.ip_forward = 1' to /etc/sysctl.d/k8s.conf and run 'sudo sysctl --system'.

Part 6 — Initialize the Control Plane

Before running kubeadm init, confirm the master node has enough memory — kubeadm requires at least ~1700 MB:

```
free -h
```

On the master node, initialize the cluster:

```
sudo kubeadm init --pod-network-cidr=192.168.0.0/16
```

A successful run ends with a message confirming the control plane initialized, followed by a kubeadm join command — copy and save this command exactly, including the token and discovery hash, as it's needed to join the worker node.

Configure kubectl access for your user:

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
kubectl get nodes # STATUS will show NotReady – this is expected until a CNI is installed
```

Part 7 — Install the Calico Network Plugin

A Kubernetes cluster needs a pod networking layer (CNI) before nodes report Ready. Install Calico on the master:

```
kubectl apply -f
https://raw.githubusercontent.com/projectcalico/calico/v3.28.0/manifests/calico.yaml
```

Wait roughly 1–2 minutes, then confirm all Calico pods are Running:

```
kubectl get pods -A
```

Part 8 — Join the Worker Node

SSH into the worker node and run the exact kubeadm join command generated during Part 6:

```
sudo kubeadm join <master-ip>:6443 \
--token <token> \
--discovery-token-ca-cert-hash sha256:<hash>
```

A successful join prints "This node has joined the cluster."

Part 9 — Verify the Cluster

```
kubectl get nodes -o wide
```

Expected output shows both nodes in Ready state — the master with role control-plane, and the worker with no role listed (a standard worker).

Part 10 — First Workload: Deploy Nginx

```
kubectl create deployment nginx --image=nginx
kubectl get pods
kubectl expose deployment nginx --type=NodePort --port=80
kubectl get svc
```

Confirm the pod reaches Running status and the service has an assigned NodePort.

Common Troubleshooting Patterns Covered in This Lab

- SSH key path errors (~ expansion differing between OS/shell) — always confirm the full literal path with 'ls -ltr'.
- Running commands from the wrong node — always check 'hostname -I' first.
- Infrastructure recreation wiping installed packages and directories — solved by using Ansible playbooks instead of manual one-off installs, so setup is repeatable.

- Container runtime not installed vs. stopped — 'systemctl status' distinguishes 'could not be found' (not installed) from 'inactive' (installed but stopped).
- kubeadm preflight failures — missing container runtime and IP forwarding are the two most common on fresh nodes.
- Insufficient node memory — kubeadm requires roughly 1700 MB minimum; verify with 'free -h' before init.

Next Labs (Building on This Cluster)

- Deploy and expose additional workloads (Services, Ingress)
- Debug CrashLoopBackOff and ImagePullBackOff pod states
- Debug Pending pods (resource requests, scheduling constraints)
- Node cordon, drain, and recovery of a failed worker
- Add readiness/liveness probes, resource requests/limits
- Persistent Volumes and PersistentVolumeClaims (e.g. backed by EBS)
- RBAC — Roles, RoleBindings, and ServiceAccounts

Cost Management Reminder

EC2 instances, Elastic IPs, and NAT gateways all incur charges outside free-tier limits. Students should:

- Stop or terminate EC2 instances when not actively working through the lab.
- Release unused Elastic IPs — an allocated but unassociated Elastic IP is billed.
- Validate and clean up resources via Terraform ('terraform destroy') or AWS CLI once a lab session is complete.
- Monitor the AWS Billing Dashboard regularly, even when using only free-tier-eligible resources.